

2026年2月11日

记下一步思路

- 1、移除项目中的pygame相关代码，不需要使用pygame来渲染
- 2、优化环境代码simple_tag_env，优化提取器context_extractor，参照以下思路：

第一步：优化环境代码 (`simple_tag_env.py`)

你的环境代码目前将“核心逻辑”与“渲染展示”严重耦合。为了让LLM读懂，我们需要把“它是怎么动的”和“它是怎么画的”分开。

建议修改点：

1. **移除冗余的 Pygame 逻辑**：将渲染相关代码（`draw`，`render`，`enable_render`）移到底部或者通过Mixin类引入，不要打断物理逻辑的阅读流。
2. **显式化语义**：在 `observation` 函数中添加注释，告诉LLM每个维度的物理含义，这对奖励设计至关重要。

这是优化后的环境结构建议（保留了核心逻辑，清理了结构）：

```
class Custom_raw_env(SimpleEnv, EzPickle):
    def __init__(self, num_good=1, num_adversaries=3, num_obstacles=2, max_cycles=100,
continuous_actions=False, render_mode=None, world_size=2.5):
        EzPickle.__init__(self, num_good, num_adversaries, num_obstacles, max_cycles,
continuous_actions, render_mode)

        # 1. 核心属性定义清晰化
        self.world_size = world_size
        self.max_force = 1.0
        self.capture_threshold = self.world_size * 0.2

        scenario = Scenario()
        world = scenario.make_world(num_good, num_adversaries, num_obstacles,
_world_size=world_size)

        SimpleEnv.__init__(self, scenario=scenario, world=world, render_mode=render_mode,
max_cycles=max_cycles, continuous_actions=continuous_actions)

        # 2. 运行时绑定（关键逻辑）
        world._env_instance = self
        self.current_actions = {}
        self.last_reward_components = {agent.name: {} for agent in world.agents}

        # 3. 空间定义（保留动态计算，但通过提取器读取运行时值）
        self._init_spaces()

    def _init_spaces(self):
        # 将空间定义逻辑独立，方便阅读
        self.action_spaces = dict()
```

```

        self.observation_spaces = dict()
        for agent in self.world.agents:
            # ... (保留原有的空间计算逻辑) ...
            # 省略具体代码以节省篇幅，逻辑保持不变
            pass

    def _execute_world_step(self):
        # 这是LLM最需要理解的物理引擎接口
        actions_by_name = {}
        for i, agent in enumerate(self.world.agents):
            action = self.current_actions[i]
            # ... (动作解算逻辑) ...
            # 这里的逻辑直接决定了力如何作用于物体，必须保留

        self.current_actions = actions_by_name
        self.world.step() # 调用动力学积分

# ... step, reset 等核心方法保持不变 ...

# =====
# 渲染模块 (LLM不需要关注的部分)
# =====
def render(self):
    pass # 具体实现折叠，LLM不需要看这几十行代码

```

第二步：重写上下文提取器 (`context_extractor.py`)

这是重头戏。我们不再使用正则表达式猜测，而是采用 **"AST解析 + 运行时反射"** 的混合模式。

- **运行时反射 (Runtime Introspection)**：直接实例化环境，打印出真实的观测空间和动作空间维度。
- **AST 清洗 (AST Cleaning)**：使用抽象语法树分析代码，自动剔除 `render`、`import pygame` 等对奖励函数设计无用的噪音，保留核心物理逻辑。

以下是为你重写的提取器：

Python

```

import ast
import inspect
import sys
import os
import importlib.util
import numpy as np

class LLMFriendlyContextExtractor:
    """
    智能环境上下文提取器 (Smart Environment Context Extractor)
    特点：
    1. 运行时反射：真实运行环境以获取准确的 space 维度。
    2. AST 代码清洗：自动移除渲染代码、非核心 import，保留物理逻辑。
    """

```

```

def __init__(self, env_file_path):
    self.file_path = env_file_path
    self.module_name = os.path.basename(env_file_path).replace('.py', '')

def get_runtime_info(self):
    """
    通过动态加载模块并实例化环境，获取真实的维度信息
    """
    spec = importlib.util.spec_from_file_location(self.module_name, self.file_path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[self.module_name] = module
    spec.loader.exec_module(module)

    # 假设环境类名为 Custom_raw_env，如果不确定可以遍历查找
    if hasattr(module, 'Custom_raw_env'):
        # 实例化一个轻量级环境
        try:
            env = module.Custom_raw_env(num_good=1, num_adversaries=3,
continuous_actions=True)
            env.reset()

            info = {
                "agent_list": env.agents,
                "observation_structure": {},
                "action_structure": {}
            }

            # 提取第一个Adversary和第一个Agent作为样本
            sample_agents = [a for a in env.agents if 'adversary' in a][:1] + \
                [a for a in env.agents if 'agent' in a][:1]

            for agent_name in sample_agents:
                obs_space = env.observation_space(agent_name)
                act_space = env.action_space(agent_name)
                info["observation_structure"][agent_name] = {
                    "shape": str(obs_space.shape),
                    "dtype": str(obs_space.dtype),
                    "low": str(np.min(obs_space.low)) if hasattr(obs_space, 'low')
else '-inf',
                    "high": str(np.max(obs_space.high)) if hasattr(obs_space, 'high')
else 'inf'
                }
                info["action_structure"][agent_name] = str(act_space)

            return info
        except Exception as e:
            return f"Runtime extraction failed: {str(e)}"
        return "Custom_raw_env class not found."

def clean_code_with_ast(self):
    """
    使用AST解析代码，移除渲染函数，保留物理逻辑
    """
    with open(self.file_path, 'r', encoding='utf-8') as f:
        source = f.read()

```

```

    tree = ast.parse(source)

    class CodeCleaner(ast.NodeTransformer):
        def visit_Import(self, node):
            # 移除 pygame, matplotlib 等绘图库
            names = [n.name for n in node.names if 'pygame' not in n.name and
'matplotlib' not in n.name]
            if not names: return None
            return node

        def visit_ClassDef(self, node):
            # 只保留 Custom_raw_env 和 Scenario
            if node.name not in ['Custom_raw_env', 'Scenario']:
                return None
            self.generic_visit(node)
            return node

        def visit_FunctionDef(self, node):
            # 移除渲染相关函数
            black_list = ['render', 'draw', 'enable_render', 'render_matplotlib',
'draw_grid_and_axes']
            if any(b in node.name for b in black_list):
                return None
            return node

    cleaner = CodeCleaner()
    cleaned_tree = cleaner.visit(tree)
    ast.fix_missing_locations(cleaned_tree)
    return ast.unparse(cleaned_tree)

    def generate_prompt_context(self):
        runtime_info = self.get_runtime_info()
        cleaned_code = self.clean_code_with_ast()

        prompt = f"""
# Environment Context for Reward Design

## 1. Runtime Spaces Analysis (Ground Truth)
This information is extracted from the running environment instance.
{runtime_info}

## 2. Core Environment Logic (Rendering stripped)
This code contains the physics, state transitions, and observation construction logic.
```python
{cleaned_code}

```

### 3. Key Physical Constants (Found in code)

- world\_size: (Check logic above)
- max\_force: (Check logic above)

- capture\_threshold: (Check logic above)

```
"""
```

```
return prompt
```

```
if name == "main":
```

```
extractor = LLMFriendlyContextExtractor("simple_tag_env.py") # 替换为你的路径
```

```
print(extractor.generate_prompt_context())
```

```
教授的指导意见 (Professor's Insights)
```

```
1. **关于代码清洗逻辑**:
```

```
在 `CodeCleaner` 类中,我特意移除了 `render`, `draw` 等函数。因为 **LLM 设计奖励函数时,不需要知道你的圆圈是用什么颜色画的**。它只需要关注 `_execute_world_step` (力如何转化为速度) 和 `observation` (看到的数据包含什么)。去除这些噪音后,你可以将完整的逻辑代码放入 Prompt,而不需要盲目截断。
```

```
2. **关于“运行时反射”的价值**:
```

```
你会发现我在 `get_runtime_info` 中并没有去解析文本,而是直接运行了代码。这是 Python 的优势。LLM 经常苦恼于 `dim_p * 2 + 1` 到底是几。现在,我们直接告诉它:“Adversary_0 的动作空间是 Box(5,)”。这消除了歧义,让 LLM 能够基于确定的输入输出维度来编写数学公式。
```

3、优化reward\_design\_agent,生成多个回复时使用如下逻辑:

```

 # 生成多个候选
 n_candidates = self.config['generation']
 n]['num_candidates']
 raw_outputs = self.llm.generate(
 prompt=prompt,
 n=n_candidates,
 temperature=self.config['generation']
 n]['temperature'],
 max_tokens=self.config['generation']
 n]['max_tokens'],
 system_message=self.prompt_builder.
 SYSTEM_MESSAGE
)

```

后续代基于父本进化时也生成多个回复,生成1个回复大概率只有1个变体(考验LLM的质量)

4、将逃跑者的奖励函数从可插拔奖励函数中移出来,不需要LLM设计逃跑者的奖励函数